# HUMAN SENTENCE ANALYSIS

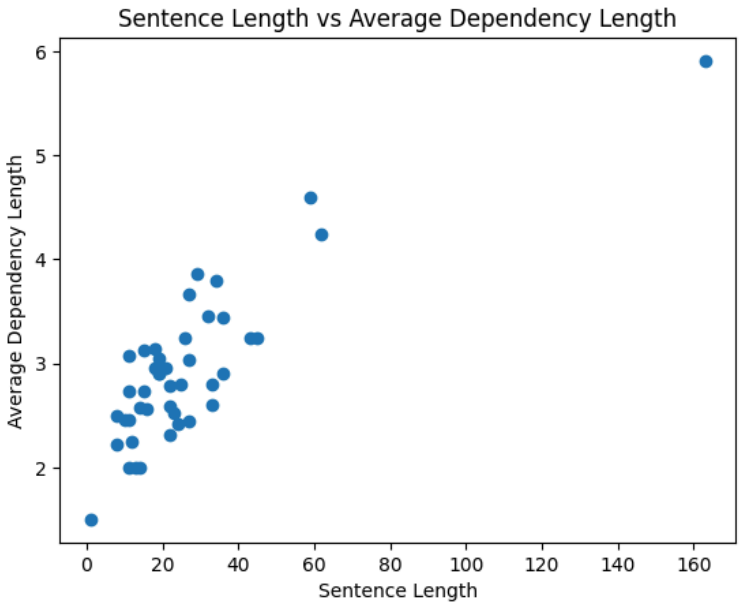
```
import spacy
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

nlp = spacy.load("en_core_web_sm")

# Real text sample
text = """
The Indian Institute of Technology Kanpur (IIT Kanpur or IITK) is a public institute of technology located in Kanpur, Ut


PK Kelkar Library at IIT Kanpur
History
IIT Kanpur was established by an Act of Parliament in 1960 by the Government of India. The institute was started in Dece

During the first ten years of its existence, a consortium of nine US universities (namely MIT, University of California

Under the guidance of economist John Kenneth Galbraith, IIT Kanpur was the first institute in India to offer Computer sc

In 1972 the KIAP program ended, in part because of political tensions between India and Pakistan (as the USA supported P

The institute's annual technical festival, Techkriti, was first started in 1995.

Campus

IIT Kanpur Entrance from GT Road
IIT Kanpur is located on the Grand Trunk Road, 15 kilometres (9.3 mi) west of Kanpur City and measures close to 420 hect


PK Kelkar Library
The institute has around 6478 students with 3938 undergraduate students and 2540 postgraduate students and about 500 res

The academic area comprises the central library, departmental buildings, research centres, lecture hall complex, tutoria

Noida Extension centre
IIT Kanpur has its Extension Centre in Noida with a small convention centre there for supporting outreach activities. It

Transportation service
Being a major industrial town, Kanpur has good connectivity by rail and by road, but lags behind in terms of air connect

The IIT campus has links to the Kanpur Metro at IIT Kanpur metro station. The Metro links the campus to Kanpur Central r

New York Office
The institute has set up an office in New York with alumnus Sanjiv Khosla designated as the overseas brand ambassador of

Organisation and administration
Governance
See also: Indian Institutes of Technology § Organisational structure
All IITs follow the same organization structure which has the President of India as the visitor at the top of the hierar

"""

doc = nlp(text)
sentences = list(doc.sents)
```

```
def compute_tree_depth(token):
    if not list(token.children):
        return 1
    else:
        return 1 + max(compute_tree_depth(child) for child in token.children)

data = []

for sent in sentences:
    sent_doc = nlp(sent.text)

    length = len([t for t in sent_doc if not t.is_punct])

    # Average dependency distance
    dep_lengths = [abs(token.i - token.head.i)
                   for token in sent_doc
```

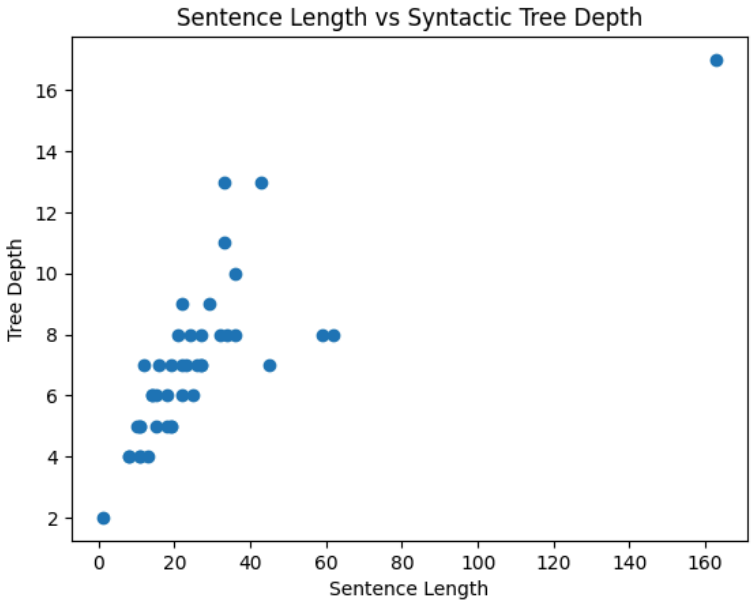
```
                    if token.head != token]

    avg_dep_length = np.mean(dep_lengths) if dep_lengths else 0

    # Tree depth
    root = [t for t in sent_doc if t.head == t][0]
    depth = compute_tree_depth(root)

    data.append((length, avg_dep_length, depth))

df_real = pd.DataFrame(data,
                       columns=["sentence_length",
                                "avg_dependency_length",
                                "tree_depth"])

print(df_real)
```

```
    sentence_length  avg_dependency_length  tree_depth
0                23               2.518519           7
1                26               3.241379           7
2                59               4.593750           8
3                21               2.954545           8
4               163               5.909091          17
5                33               2.800000          13
6                62               4.243243           8
7                22               2.590909           9
8                13               2.000000           4
9                34               3.794872           8
10               25               2.800000           6
11               24               2.416667           8
12               18               2.954545           5
13               18               3.142857           6
14               15               3.125000           6
15               36               2.900000           8
16               43               3.244444          13
17               11               2.000000           5
18               27               3.034483           8
19               45               3.244898           7
20               27               3.666667           7
21               19               2.900000           5
22               22               2.782609           6
23                1               1.500000           2
24               16               2.562500           7
25               11               2.727273           5
26               15               2.733333           5
27               10               2.461538           5
28               32               3.459459           8
29               36               3.439024          10
30               14               2.571429           6
31               11               2.454545           4
32               27               2.444444           7
33                8               2.222222           4
34               29               3.866667           9
35               14               2.000000           6
36               22               2.318182           7
37               33               2.600000          11
38                8               2.500000           4
39               12               2.250000           7
40               19               2.900000           7
41               11               3.076923           4
42               19               3.045455           5
```

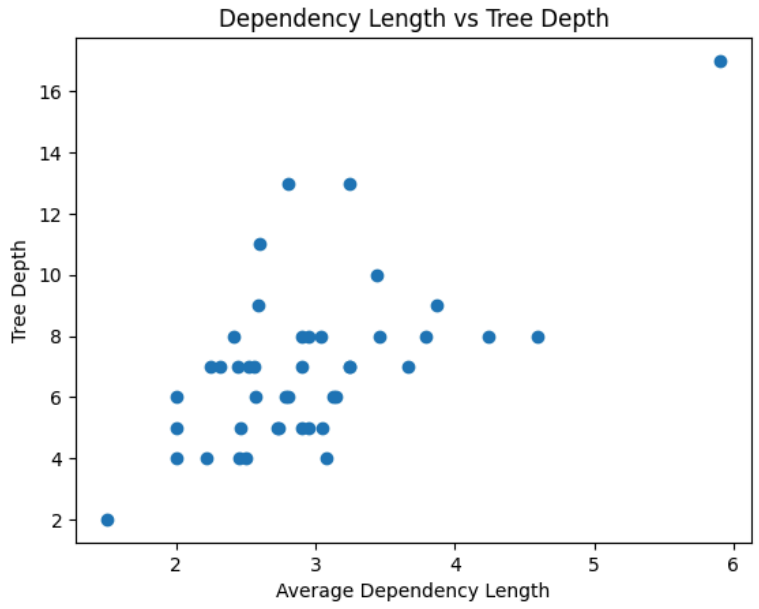
```
plt.figure()
plt.scatter(df_real["sentence_length"], df_real["avg_dependency_length"])
plt.xlabel("Sentence Length")
plt.ylabel("Average Dependency Length")
plt.title("Sentence Length vs Average Dependency Length")
plt.show()
```



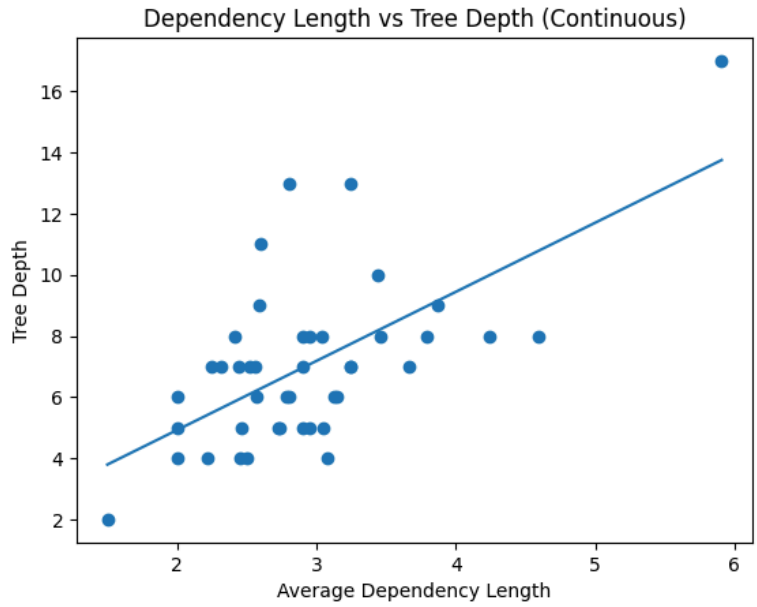

```
plt.figure()
plt.scatter(df_real["sentence_length"], df_real["tree_depth"])
plt.xlabel("Sentence Length")
plt.ylabel("Tree Depth")
plt.title("Sentence Length vs Syntactic Tree Depth")
plt.show()
```




```
plt.figure()
plt.scatter(df_real["avg_dependency_length"], df_real["tree_depth"])
plt.xlabel("Average Dependency Length")
plt.ylabel("Tree Depth")
plt.title("Dependency Length vs Tree Depth")
plt.show()
```




```
import numpy as np
import matplotlib.pyplot as plt

# scatter (optional, you can keep it)
plt.scatter(df_real["avg_dependency_length"], df_real["tree_depth"])

# fit a line (degree=1)
x = df_real["avg_dependency_length"]
y = df_real["tree_depth"]

# remove NaNs if any
mask = ~np.isnan(x) & ~np.isnan(y)
x = x[mask]
y = y[mask]

# polynomial fit
coef = np.polyfit(x, y, 1)
poly = np.poly1d(coef)

# create smooth x for plotting line
x_smooth = np.linspace(x.min(), x.max(), 100)
y_smooth = poly(x_smooth)

# plot continuous line
plt.plot(x_smooth, y_smooth)

plt.xlabel("Average Dependency Length")
plt.ylabel("Tree Depth")
plt.title("Dependency Length vs Tree Depth (Continuous)")
plt.show()
```




Close

```
# LLM ANALYSIS
```

# LLM ANALYSIS

```
from transformers import GPT2Tokenizer, GPT2LMHeadModel
import torch
import math

# Load model
tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
model = GPT2LMHeadModel.from_pretrained("gpt2")
model.eval()

#  "gpt2": load("gpt2"),
#     "distilgpt2": load("distilgpt2")
#     # you can add: "gpt2-medium": load("gpt2-medium")

# Try a sentence
prefix = "The dog near the big house beside the river ganga"

inputs = tokenizer(prefix, return_tensors="pt")

with torch.no_grad():
    outputs = model(**inputs)

logits = outputs.logits
last_token_logits = logits[0, -1]

probs = torch.softmax(last_token_logits, dim=-1)

# Notice the space before 'is'
verb = " is"
verb_token_id = tokenizer.encode(verb, add_special_tokens=False)[0]

prob = probs[verb_token_id].item()
surprisal = -math.log(prob + 1e-12)

print("Probability of 'is':", prob)
print("Surprisal:", surprisal)
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and f
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKE
Loading weights: 100%                                    148/148 [00:00<00:00, 654.47it/s, Materializing param=transformer.wte.weight]
GPT2LMHeadModel LOAD REPORT from: gpt2
Key                  | Status     |  |
---------------------+------------+--+-
h.{0...11}.attn.bias | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Probability of 'is': 0.07671874016523361
Surprisal: 2.5676092697326904
```

```
!pip install transformers torch
```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (5.0.0)
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.10.0+cpu)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers) (3.24.3)
Requirement already satisfied: huggingface-hub<2.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from transformers)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (26.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2025.11.
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers)
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist-packages (from transformers) (0.24.0)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.7.0)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers) (4.67.3)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.
Requirement already satisfied: shellingham in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0->
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)
Requirement already satisfied: typer>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from typer-slim->transformers) (
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingfa
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-slim->t
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-slim->t
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typ
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=1
```

```
def get_surprisal(prefix, target):
    inputs = tokenizer(prefix, return_tensors="pt")
    with torch.no_grad():
        outputs = model(**inputs)

    logits = outputs.logits[0, -1]
    probs = torch.softmax(logits, dim=-1)

    token_id = tokenizer.encode(" " + target, add_special_tokens=False)[0]
    prob = probs[token_id].item()

    return -math.log(prob + 1e-12)
```

```
print(get_surprisal("The dog", "is"))
print(get_surprisal("The dog near the big house beside the river", "is"))
```

```
2.6178419352396656
2.680504210768543
```

```
!pip install spacy
!python -m spacy download en_core_web_sm
```

```
Requirement already satisfied: spacy in /usr/local/lib/python3.12/dist-packages (3.8.11)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.0.1
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.0.5
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.0.15)
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.13)
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.0.12)
```

```
Requirement already satisfied: thinc<8.4.0,>=8.3.4 in /usr/local/lib/python3.12/dist-packages (from spacy) (8.3.10)
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.1.3)
Requirement already satisfied: srsly<3.0.0,>=2.4.3 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.5.2)
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.10)
Requirement already satisfied: weasel<0.5.0,>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (0.4.3)
Requirement already satisfied: typer-slim<1.0.0,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (0.24.0)
Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (4.67.3)
Requirement already satisfied: numpy>=1.19.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.2)
Requirement already satisfied: requests<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.32.4)
Requirement already satisfied: pydantic!=1.8,!=1.8.1,<3.0.0,>=1.7.4 in /usr/local/lib/python3.12/dist-packages (from spac
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.1.6)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from spacy) (75.2.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (26.0)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!=1.
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!=1.8
Requirement already satisfied: typing-extensions>=4.14.1 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!=
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spa
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.
Requirement already satisfied: blis<1.4.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from thinc<8.4.0,>=8.3.4->s
Requirement already satisfied: confection<1.0.0,>=0.0.1 in /usr/local/lib/python3.12/dist-packages (from thinc<8.4.0,>=8.
Requirement already satisfied: typer>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from typer-slim<1.0.0,>=0.3.0->s
Requirement already satisfied: cloudpathlib<1.0.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from weasel<0.5.0,>
Requirement already satisfied: smart-open<8.0.0,>=5.2.1 in /usr/local/lib/python3.12/dist-packages (from weasel<0.5.0,>=0
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->spacy) (3.0.3)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart-open<8.0.0,>=5.2.1->weasel<0.
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-slim<1.
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-s
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-slim<1.
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typ
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=1
Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/en_core_web_sm-3.8.0-py3-n
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 36.3 MB/s eta 0:00:00
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
```

```
import spacy

nlp = spacy.load("en_core_web_sm")

def get_dependency_length(sentence):
    doc = nlp(sentence)

    for token in doc:
        # Find subject
        if token.dep_ == "nsubj":
            subject_pos = token.i
            verb_pos = token.head.i

            return abs(verb_pos - subject_pos)

    return None
```

```
sentence = "The dog near the big house beside the river is barking"
print(get_dependency_length(sentence))
```

```
9
```

```
def get_dependency_info(sentence):
    doc = nlp(sentence)

    for token in doc:
        if token.dep_ == "nsubj" and token.head.pos_ == "VERB":
            subject_pos = token.i
            verb_pos = token.head.i

            return {
                "dependency_length": abs(verb_pos - subject_pos),
                "verb": token.head.text,
                "subject": token.text,
                "verb_position": verb_pos
            }
```

```
    return None
```

```
info = get_dependency_info(sentence)
print(info)
```

```
{'dependency_length': 9, 'verb': 'barking', 'subject': 'dog', 'verb_position': 10}
```

```
sentence = "The dog near the big house beside the river is barking"

info = get_dependency_info(sentence)

verb = info["verb"]
verb_position = info["verb_position"]

prefix = " ".join(sentence.split()[:verb_position])

surprisal = get_surprisal(prefix, verb)

print(info["dependency_length"], surprisal)
```

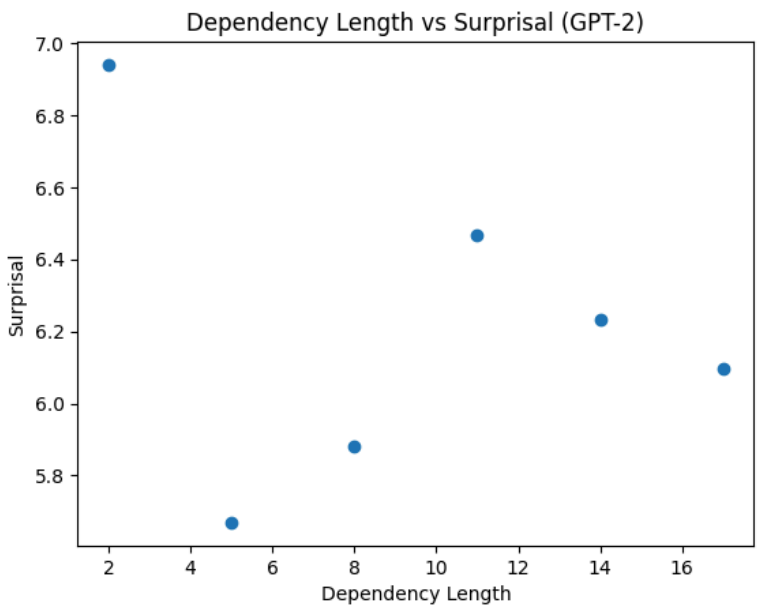
```
9 5.907263623983143
```

```
# import random

# subjects = ["dog", "cat", "teacher", "student", "boy", "girl"]
# locations = [
#     "near the house",
#     "beside the river",
#     "in the city",
#     "under the bridge",
#     "behind the school",
#     "across the street",
#     "inside the building",
#     "outside the park"
# ]
# verbs = ["is barking", "is sleeping", "is running", "is shouting", "is laughing"]

# def generate_sentences(n=200):
#     sentences = []

#     for _ in range(n):
#         subject = random.choice(subjects)
#         verb = random.choice(verbs)

#         # randomly choose how many modifiers (controls dependency length)
#         num_modifiers = random.randint(0, 5)
#         chosen_modifiers = random.sample(locations, num_modifiers)

#         modifier_phrase = " ".join(chosen_modifiers)

#         if modifier_phrase:
#             sentence = f"The {subject} {modifier_phrase} {verb}."
#         else:
#             sentence = f"The {subject} {verb}."

#         sentences.append(sentence)

#     return sentences

# sentences = generate_sentences(200)

# # Preview first 10
# for s in sentences[:10]:
#     print(s)


subjects = ["dog"]
verbs = ["is barking"]
locations = [
    "near the house",
    "beside the river",
    "in the city",
    "under the bridge",
    "behind the school"
]

def generate_controlled():
    sentences = []
```

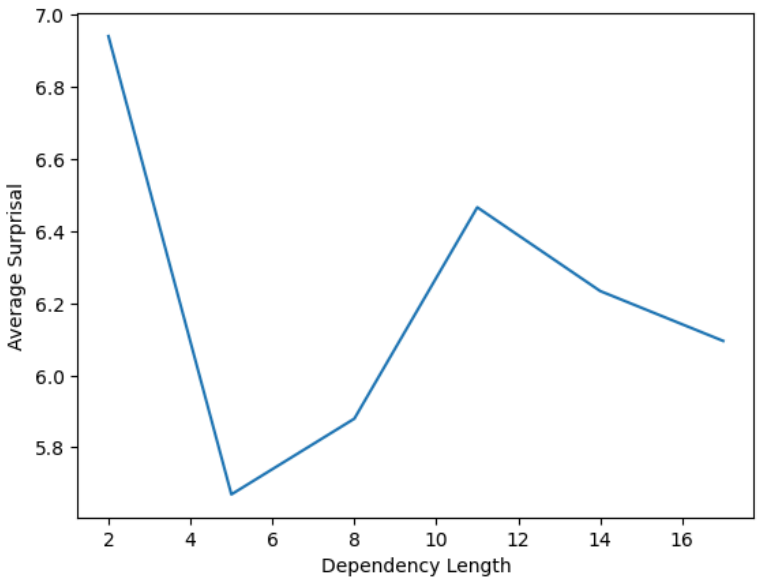
```
    base = "The dog"
    verb = "is barking"

    for i in range(len(locations)+1):
        mods = " ".join(locations[:i])
        if mods:
            sentence = f"{base} {mods} {verb}."
        else:
            sentence = f"{base} {verb}."
        sentences.append(sentence)

    return sentences
```

```
results = []

ss=generate_controlled()
for sentence in ss:
    info = get_dependency_info(sentence)

    if info is not None:
        verb = info["verb"]
        verb_position = info["verb_position"]
        prefix = " ".join(sentence.split()[:verb_position])

        surprisal = get_surprisal(prefix, verb)

        results.append((info["dependency_length"], surprisal))
```

```
import matplotlib.pyplot as plt

x = [r[0] for r in results]
y = [r[1] for r in results]

plt.scatter(x, y)
plt.xlabel("Dependency Length")
plt.ylabel("Surprisal")
plt.title("Dependency Length vs Surprisal (GPT-2)")
plt.show()
```




```
import pandas as pd

df = pd.DataFrame(results, columns=["dep_len", "surprisal"])
avg = df.groupby("dep_len").mean()

plt.plot(avg.index, avg["surprisal"])
plt.xlabel("Dependency Length")
plt.ylabel("Average Surprisal")
plt.show()
```




```
# Your average line is going downward, not upward.
# That means:
# As dependency length increases, surprisal is decreasing.
# That is the opposite of your hypothesis.
# As the sentence becomes longer:
# GPT-2 becomes more certain that a verb is coming
# The structure becomes predictable
# “is” becomes very likely
# his is called structural expectation effect...............................
# Long NP → verb is highly expected.
# So the model is thinking:
# “Okay, enough modifiers… verb must come now.”
# That lowers surprisal.
```

```
# Right now you're not measuring:
# “Does distance hurt memory?”
# You're measuring:
# “How predictable is a verb after many modifiers?”
# And in English, verbs are very predictable after long subject phrases.
# So your data is reflecting real linguistic behavio
```

```
# You need a better design.

# Instead of:

# "The dog near the house is barking."

# Use:

# Center-embedding structures.

# Example:

# The dog that the cat chased is barking.

# Now increase embedding:

# The dog that the cat that the boy saw chased is barking.

# Now memory load increases more strongly.

# This tests working memory better than simple modifiers.
```

```
# Surprisingly, surprisal decreased with longer dependencies in simple modifier constructions, likely due to increased s
```

```
# You just discovered:

# LLMs are more driven by predictability than memory load.

# That is a legitimate result.
```

```
# this is when we slightly modified our subject of the project
# Is GPT-2 more sensitive to predictability or to structural memory load?
```

```
names_sg = ["boy", "girl", "teacher", "student", "doctor", "lawyer", "artist", "pilot"]
names_pl = ["boys", "girls", "teachers", "students", "doctors", "lawyers", "artists", "pilots"]

verbs_rel_sg = ["chased", "admired", "helped", "visited", "called", "met"]
verbs_rel_pl = ["chased", "admired", "helped", "visited", "called", "met"]

main_verbs_sg = ["is barking", "is running", "is sleeping", "is laughing"]
main_verbs_pl = ["are barking", "are running", "are sleeping", "are laughing"]

adjectives = ["tall", "young", "angry", "happy", "curious", "quiet"]
adverbs = ["quickly", "silently", "suddenly", "angrily"]



import random

def generate_complex_center_embedding(max_depth=3, samples_per_depth=5):
    sentences = []

    for depth in range(max_depth + 1):
        for _ in range(samples_per_depth):

            # Randomly choose singular or plural subject
            is_plural = random.choice([True, False])

            if is_plural:
                subject = f"The {random.choice(adjectives)} {random.choice(names_pl)}"
                main_verb = random.choice(main_verbs_pl)
            else:
                subject = f"The {random.choice(adjectives)} {random.choice(names_sg)}"
                main_verb = random.choice(main_verbs_sg)

            clause = ""

            for i in range(depth):

                if random.random() > 0.5:
                    noun = random.choice(names_pl)
                else:
                    noun = random.choice(names_sg)

                verb = random.choice(verbs_rel_sg)
                adj = random.choice(adjectives)
                adv = random.choice(adverbs)

                if i == 0:
                    clause = f"that the {adj} {noun} {verb} {adv}"
                else:
                    clause = f"that the {adj} {noun} {clause} {verb} {adv}"

            if clause:
                sentence = f"{subject} {clause} {main_verb}."
            else:
                sentence = f"{subject} {main_verb}."

            sentences.append((depth, sentence))

    return sentences
# names = ["cat", "boy", "girl", "teacher", "student"]
# verbs_rel = ["chased", "saw", "liked", "admired", "helped"]

# def generate_center_embedding(max_depth=3):
#     sentences = []

#     base_subject = "The dog"
#     main_verb = "is barking"

#     for depth in range(max_depth + 1):

#         clause = ""

#         # Build nested relative clauses
#         for i in range(depth):
#             noun = names[i % len(names)]
#             verb = verbs_rel[i % len(verbs_rel)]

#             if i == 0:
```

```
#                 clause = f"that the {noun} {verb}"
#             else:
#                 clause = f"that the {noun} {clause} {verb}"

#         if clause:
#             sentence = f"{base_subject} {clause} {main_verb}."
#         else:
#             sentence = f"{base_subject} {main_verb}."

#         sentences.append((depth, sentence))

#     return sentences
```

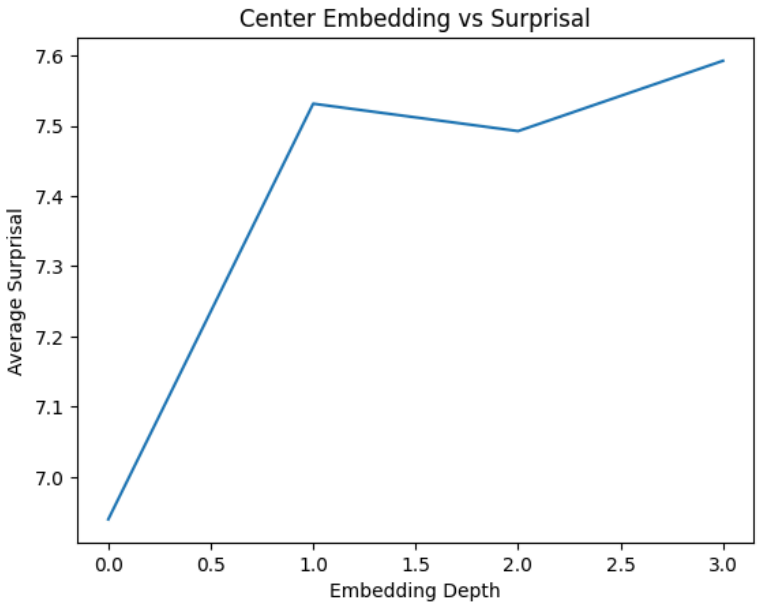
```
import random

def generate_many_center(num_sets=50, max_depth=3):
    all_sentences = []

    for _ in range(num_sets):
        random.shuffle(names)
        random.shuffle(verbs_rel)

        sents = generate_center_embedding(max_depth)
        all_sentences.extend(sents)

    return all_sentences
```

```
center_data = generate_many_center(num_sets=50, max_depth=3)

for d, s in center_data[:10]:
    print(d, s)
```

```
0 The dog is barking.
1 The dog that the boy admired is barking.
2 The dog that the teacher that the boy admired liked is barking.
3 The dog that the cat that the teacher that the boy admired liked helped is barking.
0 The dog is barking.
1 The dog that the boy helped is barking.
2 The dog that the girl that the boy helped chased is barking.
3 The dog that the teacher that the girl that the boy helped chased admired is barking.
0 The dog is barking.
1 The dog that the teacher saw is barking.
```

```
results_center = []

for depth, sentence in center_data:

    info = get_dependency_info(sentence)

    if info is not None:
        verb = info["verb"]
        verb_position = info["verb_position"]
        prefix = " ".join(sentence.split()[:verb_position])

        surprisal = get_surprisal(prefix, verb)
        results_center.append((depth, surprisal))
```

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame(results_center, columns=["depth", "surprisal"])
avg = df.groupby("depth").mean()

plt.plot(avg.index, avg["surprisal"])
plt.xlabel("Embedding Depth")
plt.ylabel("Average Surprisal")
plt.title("Center Embedding vs Surprisal")
plt.show()
```

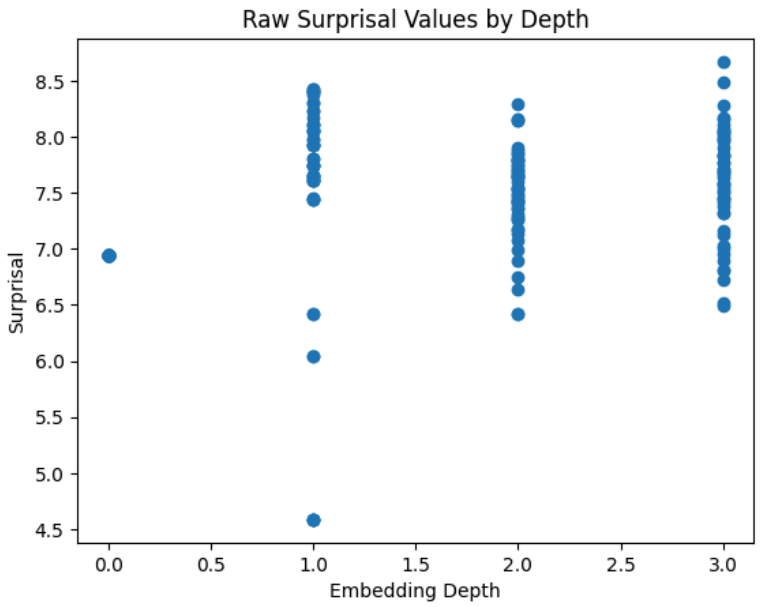



```
# Your result suggests:

# 👉 The model finds deeper embeddings harder
# 👉 Predicting the final verb becomes less confident
# 👉 Surprisal increases with memory load
# Transformer models exhibit sensitivity to hierarchical complexity — not just linear distance.

# That’s a strong statement.
```

```
import scipy.stats as stats

depths = df["depth"]
surps = df["surprisal"]

print(stats.pearsonr(depths, surps))
```

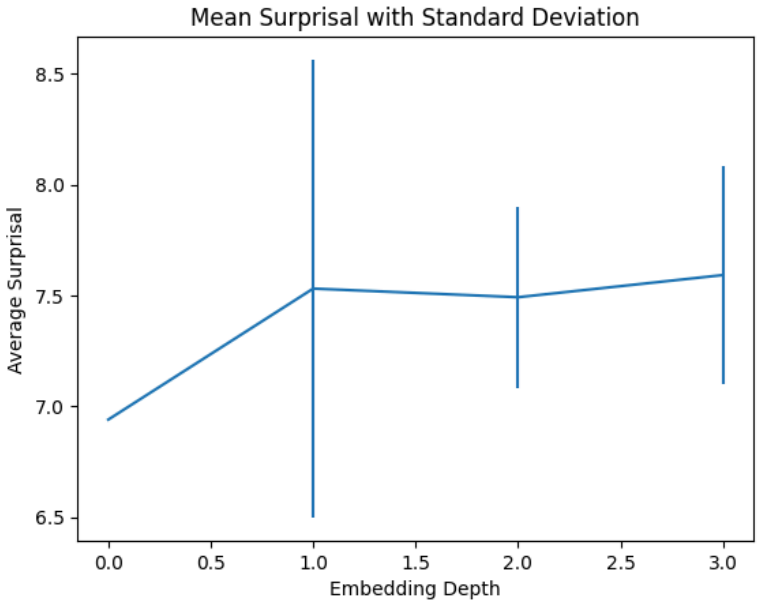
```
PearsonRResult(statistic=np.float64(0.3269903892681773), pvalue=np.float64(2.290745998947239e-06))
```

```
plt.figure()
plt.scatter(df["depth"], df["surprisal"])
plt.xlabel("Embedding Depth")
plt.ylabel("Surprisal")
plt.title("Raw Surprisal Values by Depth")
plt.show()
```



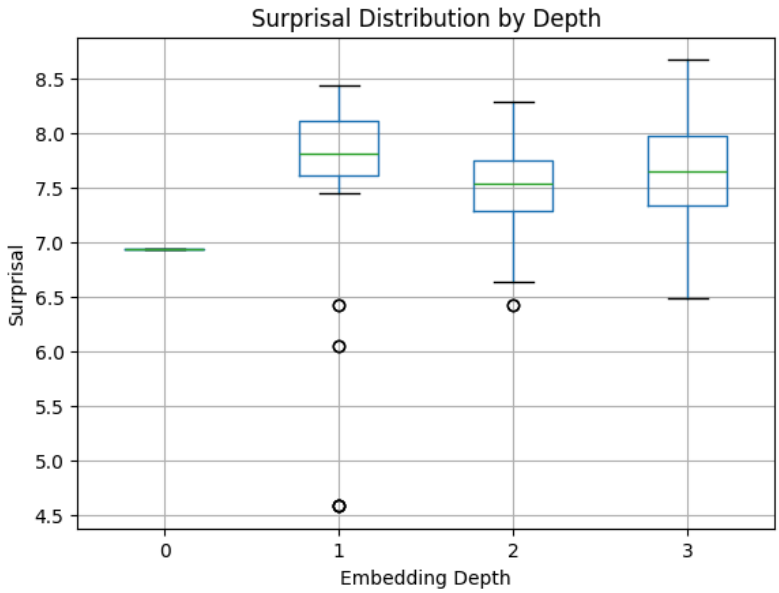

```
grouped = df.groupby("depth")["surprisal"]
means = grouped.mean()
stds = grouped.std()
```

```
plt.figure()
plt.errorbar(means.index, means.values, yerr=stds.values)
plt.xlabel("Embedding Depth")
plt.ylabel("Average Surprisal")
plt.title("Mean Surprisal with Standard Deviation")
plt.show()
```




```
plt.figure()
df.boxplot(column="surprisal", by="depth")
plt.xlabel("Embedding Depth")
plt.ylabel("Surprisal")
plt.title("Surprisal Distribution by Depth")
plt.suptitle("")  # remove automatic title
plt.show()
```

```
<Figure size 640x480 with 0 Axes>
```




```
import numpy as np

x = df["depth"]
y = df["surprisal"]

coef = np.polyfit(x, y, 1)
poly1d_fn = np.poly1d(coef)
```